HPI Hasso Plattner Institut
Digital Engineering · Universität Potsdam

Processes & threads cheat sheet

Practical 02 | Chapters 2 and 3 in the browser Linux, editors and Python in your Codespace (folder labs/practical-02).

01 Terminal editors

nano

Type directly. Shortcuts are listed at the bottom (^ = Ctrl).

```
nano work/hello.py
```

Ctrl+O, Enter	save the file
Ctrl+X	exit nano

vim

Starts in Normal mode: keys are commands, not text.

i	Insert mode: now you can type
Esc	back to Normal mode (press twice if lost)
:wq Enter	write the file and quit
:q! Enter	quit and throw the changes away

02 Look at processes

ps

List processes; `-o` picks the columns. Filter with grep.

```
ps -o pid,ppid,user,comm
ps -o pid,args | grep monitor
```

$$ / $PPID / pidof

PID of your shell, PID of its parent, PID of a program by name.

```
echo $$ $PPID
pidof monitor
```

top

Busiest processes first (%CPU). Live view: press q to quit.

```
top -b -n 1 | head -n 12
```

-b -n 1: one snapshot (batch mode), needed in the browser Linux.

/proc/PID/status

What the kernel knows: State (R running, S sleeping), PPid, Threads.

```
grep -E 'State|PPid|Threads' /proc/$$/status
```

03 Start and stop

& / $! / jobs / fg

Run in the background, PID of the last background job, list jobs, bring the latest one back.

```
monitor bed-03 &
echo $!
jobs
fg              fg %2 picks job 2
```

In a real terminal: Ctrl+Z pauses the foreground program, bg continues it in the background.

kill / kill -9

Command	Signal	What the program can do
kill PID	SIGTERM (15)	catch it, save, clean up, exit
kill -9 PID	SIGKILL (9)	nothing: removed at once

Before you send a signal

Check the PID and the full command line with `ps -o pid,args`. Polite first, -9 only if it does not react. Stop only processes you started yourself; `killall` stops every process with that name.

04 Threads for waiting

A thread is a strand of work inside a process; all threads share its memory.

```
import threading, time

start = time.perf_counter()
threads = [threading.Thread(target=read_sensor, args=(name,))
           for name in sensors]
for t in threads:
    t.start()                  # all start, nobody waits yet
for t in threads:
    t.join()                   # now wait for every thread
elapsed = time.perf_counter() - start
print(f"Total time: {elapsed:.2f} s")
```

Pitfalls: `target=read_sensor`, not `read_sensor(name)` · args is a tuple: `(name,)` · join() in the first loop makes it sequential again.

05 Processes for calculating

The GIL lets only one thread at a time run Python code. Separate processes have their own interpreter and can use several cores.

```
from concurrent.futures import ProcessPoolExecutor

if __name__ == "__main__":     # workers import this file
    with ProcessPoolExecutor(max_workers=4) as pool:
        results = list(pool.map(count_primes, BLOCKS))
```

Kind of work	Example	Tool in Python
I/O-bound	waiting for sensors, network, disk	threads
CPU-bound	prime numbers, model training	processes
light	printing a status line	keep it simple

Speedup = old time ÷ new time. 6.0 s ÷ 2.0 s = 3.0. With processes, the best possible speedup is the number of cores.

06 Shared data and locks

Critical section: from reading shared data to writing it back. Without a lock, two threads can both read 10 and both write 11.

```
lock = threading.Lock()        # once, outside the function

def admit():
    global admitted
    with lock:                 # one thread at a time
        value = admitted
        admitted = value + 1
```

Deadlock: two threads each hold one lock and wait for the other. Rule: every thread takes the locks in the same order.

07 Lab helper in the Codespace

```
cd labs/practical-02
python3 p02.py start           copy starter files to work/
python3 p02.py check threads   run, check, result code
python3 p02.py reset FILE      starter version back
```

Tasks: hello, vim, sequential, threads, cpu-threads, cpu, lock, ward. Paste the line starting with P02- into the practical page.